

2. Runtime Data Area 와 Execution Engine

JVM의 런타임 데이터 영역은 크게 **모든 스레드가 공유하는 영역**과 **스레드마다 개별로 갖는 영역**으로 나뉜다.

1. 스레드 공유 영역

1.1 Heap — GC가 관리하는 곳

모든 클래스 인스턴스와 배열을 위한 메모리가 할당되는 부분으로, 모든 JVM 스레드가 공유한다.

- 객체는 명시적으로 해제되지 않으며, **GC에 의해 회수**된다. (GC 시리즈에서 다룬 컬렉터들이 관리하는 곳이 바로 이 Heap이다.)
- Heap 용량이 부족하면 **OutOfMemoryError**가 발생한다.

1.2 Method Area

실행 시점 상수 풀(runtime constant pool), 필드 및 메서드 데이터, 생성자를 위한 코드 등 **클래스별 구조**를 저장한다. 이 영역 역시 모든 스레드가 공유한다.

2. 스레드 개별 영역

2.1 PC Register

- 각 JVM 스레드마다 하나씩 갖는 프로그램 카운터.
- 현재 실행 중인 바이트코드 명령의 주소를 담고 있다.

2.2 JVM 스택

- 스레드가 생성될 때 함께 만들어지는 개별 스택.
- **Frame**을 저장하는 공간이다.
- 메서드가 호출될 때마다 새 Frame이 생성되어 이 스택에 쌓이고(push), 메서드가 끝나면 제거된다(pop).

Frame의 구성

- **지역 변수 배열**: 메서드 파라미터, 지역 변수 등을 저장
- **피연산자 스택(operand stack)**
- **런타임 상수 풀에 대한 참조**

Frame이 지나치게 깊게 쌓이면 **StackOverflowError**가 발생한다.

2.3 Native Method Stack

Java가 아닌 다른 언어로 작성된 **네이티브 메서드**를 지원하기 위한 스택이다.

3. 실행 엔진 (Execution Engine)

메모리에 올라간 바이트코드를 **실제로 실행하는 부분**이다.

3.1 인터프리터 (Interpreter)

JVM이 시작되면 표준 인터프리터가 바이트코드를 한 줄씩 읽어, 그때그때 해석해서 실행한다.

3.2 JIT 컴파일러

- 애플리케이션이 실행되는 동안 코드를 분석해 성능 병목을 찾아내고, **성능에 결정적인 부분만 골라 기계어로 컴파일**해 성능을 올린다.
- 거의 쓰이지 않는 코드는 컴파일하지 않는다.
- 자주 실행되는 부분만 골라 캐싱해 재사용한다.

JIT 컴파일러에는 두 종류가 있다.

- **C1 (클라이언트 컴파일러)**: 빠르게 컴파일하지만 최적화 수준이 낮다.
- **C2 (서버 컴파일러)**: 컴파일 시간이 느리지만 최적화 수준이 높다.

현대 HotSpot은 이 둘을 함께 쓰는 **계층형 컴파일(Tiered Compilation)** 을 사용한다. 처음에는 인터프리터로 실행하다가, 어느 정도 자주 실행되면 C1으로 빠르게 컴파일하고, 그중에서도 가장 뜨거운(hot) 코드만 C2로 다시 컴파일해 최고 수준으로 최적화한다.

